

Fundamentals of Data Science

Lecture 9 - Model tracking, reproducibility and deployment

Adrian Ochs ¹ and Christian Roerig ²

Michaelmas 2025

¹Co-founder @ Quantum Leap, ex-QuantCo

²Data Scientist/ML Engineer @ QuantCo

Table of contents

1. Model tracking with MLFlow

- What is MLFlow?

- Storing models and artifacts

2. Reproducibility

- Reproducing via conda environment

- Version independence via ONNX format

3. Deploying a model

- Considerations for production

- Simple model deployment example

Model tracking with MLFlow



Keeping track of your models

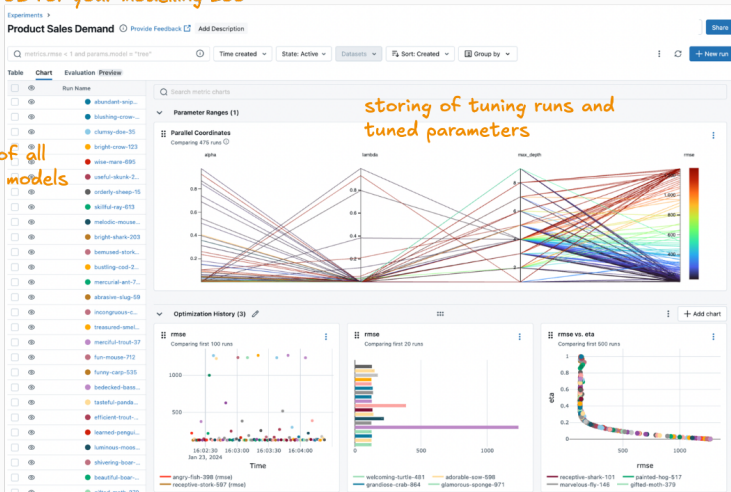
Keeping track of our models helps us in various aspects of our work as data scientists:

- **Reproducibility:** Captures precise experiment details for reconstruction.
- **Performance Monitoring:** Tracks model performance and drift over time.
- **Collaboration:** Enables transparent, shareable model development.
- **Experiment Management:** Structures complex ML workflows systematically.
- **Deployment Governance:** Maintains model lineage for auditing.
- **Optimization:** Provides empirical insights for rapid improvement.

What is MLFlow?

UI for your modelling zoo

library of all
trained models



comparing performance between model runs

What is MLFlow?

- [MLFlow](#) is open source and offers a backend and frontend for model storing and tracking.
- It also offers support for deploying models, but most corporates have a custom deployment process and quality gates.
- It allows us to store runs with all their artifacts and metadata to be able to keep track of different model runs and to reproduce them.
- Furthermore, we can compare metrics between different model runs to have a clear improvement path.

How to log a model to MLFlow

```
1  import lightgbm as lgb
2  import mlflow
3  import mlflow.lightgbm
4  from sklearn.datasets import make_regression
5  from sklearn.model_selection import train_test_split
6  from sklearn.metrics import mean_squared_error, r2_score
7
8  # Generate synthetic regression data
9  X, y = make_regression(n_samples=1000, n_features=20, noise=0.1, random_state=42)
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
11
12 # MLflow experiment tracking
13 mlflow.set_experiment("LightGBM Regression")
14
15 with mlflow.start_run():
16     # Train LightGBM regressor
17     model = lgb.LGBMRegressor(n_estimators=100, random_state=42)
18     model.fit(X_train, y_train)
19
20     # Predictions
21     y_pred = model.predict(X_test)
22
23     # Regression metrics
24     mse = mean_squared_error(y_test, y_pred)
25     r2 = r2_score(y_test, y_pred)
26
27     # Log metrics
28     mlflow.log_metrics({
29         "mean_squared_error": mse,
30         "r2_score": r2
31     })
32     # Log model parameters
33     mlflow.log_params(model.get_params())
34
35     # Log the model
36     mlflow.lightgbm.log_model(model, "regression_model")
```

How to store generic artifacts to MLFlow

We can also store any kind of evaluation to the run, using

`log_artifacts`.

```
# Create predicted vs actual plot
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Predicted vs Actual Values')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')

# Save plot
plt.tight_layout()
plt.savefig('predicted_vs_actual.png')

# Log plot as artifact
mlflow.log_artifact('predicted_vs_actual.png')
```


Running MLFlow locally

MLFlow saves the models by default in a local `mlruns` folder. Within the same directory, we can simply run `mlflow ui`, to start MLFlow locally. If we saved our runs somewhere else, we have to provide the path as `--tracking-uri /path/to/model/folder`.

The screenshot displays the MLFlow web interface. The top navigation bar includes the MLFlow logo (version 2.18.0), tabs for 'Experiments' and 'Models', and links for 'GitHub' and 'Docs'. On the left, the 'Experiments' sidebar shows a search bar and a list of experiments, with 'LightGBM Regression' selected. The main panel is titled 'LightGBM Regression' and includes options to 'Provide Feedback' and 'Add Description'. Below the title, there are tabs for 'Runs', 'Evaluation', 'Experimental', and 'Traces'. The 'Runs' tab is active, showing a search bar with the query 'metrics.rmse < 1 and params.model = "tree"', filters for 'Time created' and 'State: Active', and a '+ New run' button. A table below lists the runs, with one run named 'gifted-sponge-273' shown, created '17 minutes ago' with a duration of '4.6s'. The table headers are 'Run Name', 'Created', 'Dataset', 'Duration', 'Source', and 'Models'.

Run Name	Created	Dataset	Duration	Source	Models
● gifted-sponge-273	17 minutes ago	-	4.6s	mlflow...	lightgbm

Dive into the MLFlow UI

Overview

Model metrics

System metrics

Artifacts

Description

No description

Details

Created at	2024-12-07 09:14:23
Created by	christianrorig
Experiment ID	503271943587891896
Status	Finished
Run ID	22eb5cd95b8d47908f8feaf49640551e
Duration	5.0s
Datasets used	—
Tags	Add
Source	ipykernel_launcher.py
Logged models	lightgbm
Registered models	—

Parameters (19)

Q Search parameters

Parameter	Value
reg_lambda	0.0
boosting_type	gbdt
subsample_for_bin	200000
max_depth	-1

Metrics (2)

Q Search metrics

Metric	Value
r2_score	0.9029062153087825
mean_squared_error	3635.4392908543314

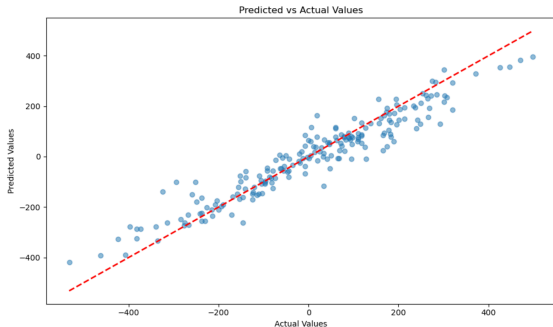
Dive into the MLFlow UI

Overview Model metrics System metrics **Artifacts**

regression_model
MLmodel
conda.yaml
model.pkl
python_env.yaml
requirements.txt
predicted_vs_actual.png

predicted_vs_actual.png 50.66KB

Path: file:///Users/christianorig/Code/teaching/solution_ps3/mlruns/503271943587891896/22eb5cd95b8d47908f8feaf49640551e/artifacts/predicted_vs_ac



Predict from stored model

Instructions on how to predict from stored model are also listed in the `MLFlow` UI under the model folder.

```
run_id = "22eb5cd95b8d47908f8feaf49640551e"  
model_uri = f"runs:{run_id}/regression_model"  
model = mlflow.lightgbm.load_model(model_uri)
```

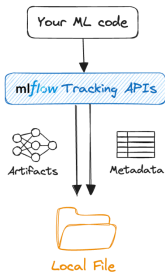
```
model.predict(X_test)[:5]
```

✓ 0.0s

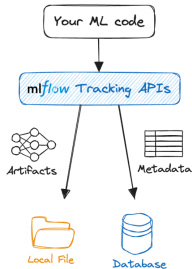
```
array([-230.45533868, -47.20336174, -79.98298855, 145.80559482,  
       171.97252672])
```

Common MLFlow Setups

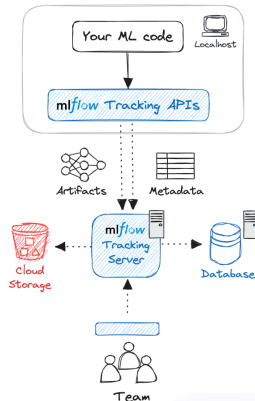
1. Localhost (default)



2. Localhost w/ various data stores



3. Remote Tracking w/ Tracking Server



Source: [MLFlow docs](#)

Reproducibility



The importance of reproducibility

Reproducibility of our trained models is crucial:

- **Collaboration:** Colleagues need to be able to reproduce our results.
- **Debugging:** Being able to reproduce results helps with debugging as it establishes a ground truth.
- **Production:** Models are often running in production for years and are expected to work consistently.

Why your code alone does not guarantee reproducibility

Although active projects try to minimize breaking changes, they are a reality.

```
import numpy as np
import sklearn
from sklearn.ensemble import RandomForestRegressor
from sklearn.datasets import make_regression

# Generate synthetic regression data
X, y = make_regression(n_samples=100, n_features=4, noise=0.1)

# Fit RandomForestRegressor
rf = RandomForestRegressor(n_estimators=10, random_state=42)
rf.fit(X, y)

print("Scikit-learn version:", sklearn.__version__)
# Pre-1.4 approach
try:
    base_estimator = rf.base_estimator_
    print("Pre-1.4 access: ", base_estimator)
except AttributeError:
    print("base_estimator_ no longer available")

# Post-1.4 approach
estimator = rf.estimator_
print("Post-1.4 access: ", estimator)

✓ 0.0s

Scikit-learn version: 1.5.2
base_estimator_ no longer available
Post-1.4 access: DecisionTreeRegressor()
```


Ways to persist your model

Once we have a trained model, we typically want to persist it, so that others, including our future self can use it too. We can split the options into **environment-dependent** and **environment-independent**:

Environment-dependent:

- E.g. `pickle`, `cloudpickle`
- Needs Python environment
- No guarantee that model works as expected if we do not use exact same environment as in training (see `conda-lock`)

Environment-independent:

- `ONNX` format
- No need for Python
- Binary serialization of the model making it independent of any package versions

Reproduce your model training environment

Compare the information in the following two environment files:

```
environment.yml
1  name: ps3
2  channels:
3    - conda-forge
4    - nodefaults
5  dependencies:
6    # defaults
7    - python>=3.8
8    - pip
9    - pre-commit
10   - pytest>=6 # Adds --import-mode option
11   - pytest-cov
12   - setuptools-scm
13   # data
14   - pandas
15   - numpy<2
16   - dask-ml
17   - ydata-profiling
18   # interactive
19   - ipykernel
20   # plots
21   - matplotlib
22   - nbformat
23   # ml
24   - glum
25   - lightgbm
26   - scikit-learn
27   - dalex
```

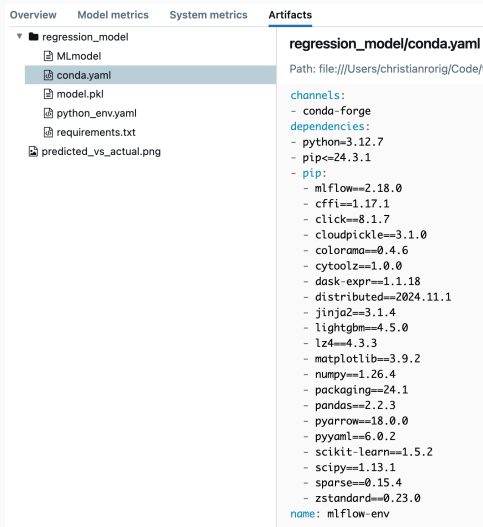
environment.yml

```
conda-lock.yml
13466 - name: s2n
13470 dependencies:
13474 url: https://conda.anaconda.org/conda-forge/linux-64/s2n-1.5.9-h0fd0e...
13475 hash:
13476   md5: f472432f3753c5ca763d2497e2ea30bf
13477   sha256: f2c8e55d6caa8d87a482b1f133963c184de1ccb2303b77cc8ca06c79425...
13478 category: main
13479 optional: false
13480 - name: scikit-learn
13481 version: 1.5.2
13482 manager: conda
13483 platform: linux-64
13484 dependencies:
13485   __glibc: '>=2.17,<3.0.a0'
13486   _openmp_mutex: '>=4.5'
13487   joblib: '>=1.2.0'
13488   libgcc: '>=13'
13489   libstdcxx: '>=13'
13490   numpy: '>=1.19,<3'
13491   python: '>=3.12,<3.13.0a0'
13492   python_abi: 3.12.*
13493   scipy: ''
13494   threadpoolctl: '>=3.1.0'
13495 url: https://conda.anaconda.org/conda-forge/linux-64/scikit-learn-1.5...
13496 hash:
13497   md5: 6b5f4c68483bd0c22bca9094dafc606b
13498   sha256: 3118b687c7cfb4484cc5c65591b611d8343ea2424cb75e1e0b0980d0de...
```

conda-lock.yml

MLFlow stores a pinned environment.yml for us

When logging a model, MLFlow stores the specific environment used for training, so we can reproduce the model later on.



The screenshot displays the MLFlow web interface with the 'Artifacts' tab selected. On the left, a file tree under 'regression_model' lists 'MLModel', 'conda.yaml' (highlighted), 'model.pkl', 'python_env.yaml', 'requirements.txt', and 'predicted_vs_actual.png'. The right pane shows the content of 'regression_model/conda.yaml', including its file path and a list of pinned dependencies for both conda and pip.

```
regression_model/conda.yaml
Path: file:///Users/christianrorig/Code/

channels:
- conda-forge
dependencies:
- python=3.12.7
- pip<=24.3.1
- pip:
  - mlflow==2.18.0
  - cffi==1.17.1
  - click==8.1.7
  - cloudpickle==3.1.0
  - colorama==0.4.6
  - cytoolz==1.0.0
  - dask-expr==1.1.18
  - distributed==2024.11.1
  - jinja2==3.1.4
  - lightgbm==4.5.0
  - lz4==4.3.3
  - matplotlib==3.9.2
  - numpy==1.26.4
  - packaging==24.1
  - pandas==2.2.3
  - pyarrow==18.0.0
  - pyyaml==6.0.2
  - scikit-learn==1.5.2
  - scipy==1.13.1
  - sparse==0.15.4
  - zstandard==0.23.0
name: mlflow-env
```

What is ONNX?

Instead of keeping track of Python environments, we can convert the model to the ONNX format.



Definition:

- Open-source file format for ML models
- Enables cross-framework interoperability
- Supports model transfer between frameworks

Key Benefits:

- Simplifies model deployment
- Reduces framework lock-in
- Optimizes model performance

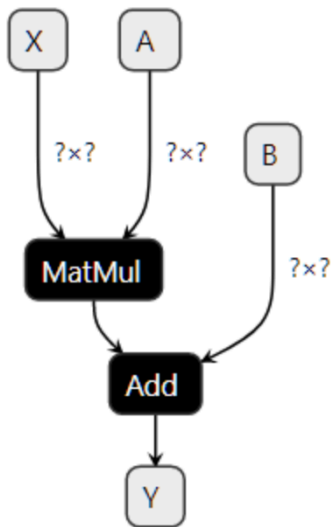
Key concepts of ONNX

ONNX can be compared to a programming language specialized in mathematical functions. It defines all the necessary operations a machine learning model needs to implement its inference function with this language.

```
def onnx_linear_regressor(X):  
    "ONNX code for a linear regression"  
    return onnx.Add(onnx.MatMul(X, coefficients), bias)
```

Source: [ONNX docs](#)

The graph structure of ONNX



Building an ONNX graph - part I

```
# imports

from onnx import TensorProto
from onnx.helper import (
    make_model, make_node, make_graph,
    make_tensor_value_info)
from onnx.checker import check_model

# inputs

# 'X' is the name, TensorProto.FLOAT the type, [None, None] the shape
X = make_tensor_value_info('X', TensorProto.FLOAT, [None, None])
A = make_tensor_value_info('A', TensorProto.FLOAT, [None, None])
B = make_tensor_value_info('B', TensorProto.FLOAT, [None, None])

# outputs, the shape is left undefined

Y = make_tensor_value_info('Y', TensorProto.FLOAT, [None])

# nodes

# It creates a node defined by the operator type MatMul,
# 'X', 'A' are the inputs of the node, 'XA' the output.
node1 = make_node('MatMul', ['X', 'A'], ['XA'])
node2 = make_node('Add', ['XA', 'B'], ['Y'])
```

Building an ONNX graph - part II

```
# from nodes to graph
# the graph is built from the list of nodes, the list of inputs,
# the list of outputs and a name.

graph = make_graph([node1, node2], # nodes
                   'lr', # a name
                   [X, A, B], # inputs
                   [Y]) # outputs

# onnx graph
# there is no metadata in this case.

onnx_model = make_model(graph)

# Let's check the model is consistent,
# this function is described in section
# Checker and Shape Inference.
check_model(onnx_model)

# the work is done, let's display it...
print(onnx_model)
```


Conversion of scikit-learn model to ONNX

```
from sklearn.linear_model import LinearRegression
from sklearn.datasets import make_regression
import numpy as np
from skl2onnx import to_onnx
import onnx
```

← helper function to convert scikit-learn pipelines to ONNX

```
# Generate some random data for regression
X, y = make_regression(n_samples=100, n_features=1, noise=0.1, random_state=42)

# Train a linear regression model in Python
model = LinearRegression()
model.fit(X, y)

# Convert the sklearn model to ONNX format
onnx_model = to_onnx(model, X)
onnx.save_model(onnx_model, 'linear_regression_model.onnx')
```

← convert and save ONNX model

Conversion of scikit-learn model to ONNX

```
onnx_model
```

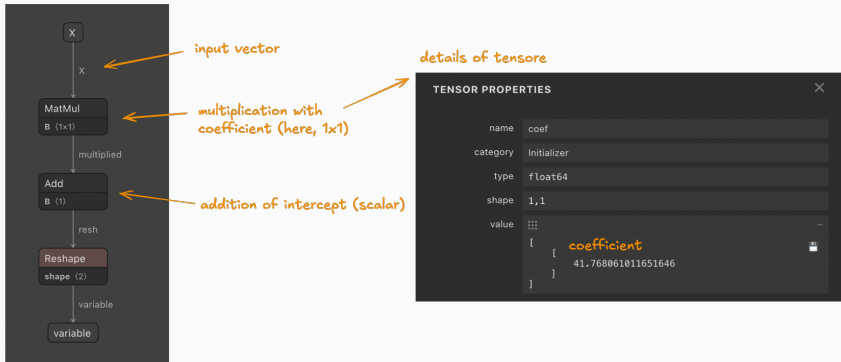
```
✓ 0.0s
```

```
ir_version: 10
producer_name: "skl2onnx"
producer_version: "1.17.0"
domain: "ai.onnx"
model_version: 0
doc_string: ""
graph {
  node {
    input: "X"
    input: "coef"
    output: "multiplied"
    name: "MatMul"
    op_type: "MatMul"
    domain: ""
  }
  node {
    input: "multiplied"
    input: "intercept"
    output: "resh"
    name: "Add"
    op_type: "Add"
    domain: ""
  }
  node {
    input: "resh"
  }
  ...
}
```

metadata information

details of ONNX graph

Conversion of scikit-learn model to ONNX



visualization by netron.app

Deploying a model



Considerations for production

Performance and Accuracy

- Validate model on unseen data
- Ensure consistent accuracy across scenarios

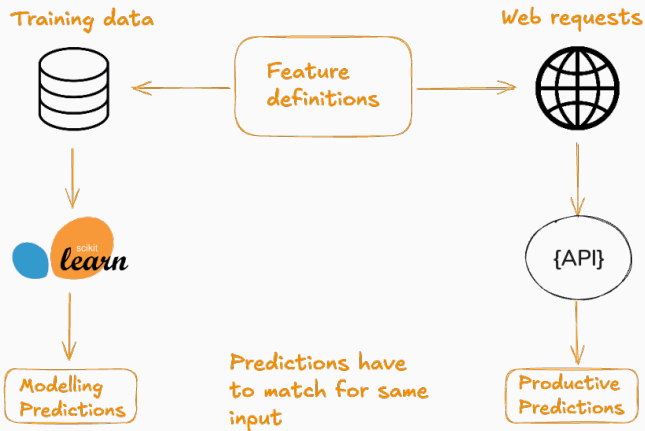
Scalability and Infrastructure

- Design robust computational setup
- Optimize for latency and throughput

Monitoring and Maintenance

- Implement real-time performance tracking
- Detect data drift and model degradation

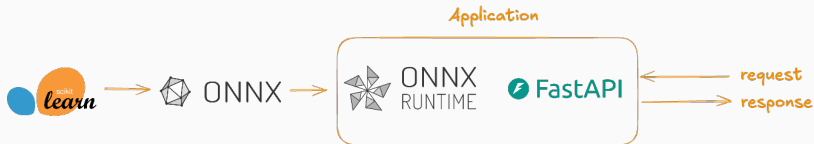
Test that your model behaviour is the same in production



Simple model deployment example

To illustrate how a very simplistic model deployment could look like, we:

1. Build a simple linear regression model
2. Convert the model to ONNX
3. Create validation classes for Input and Output
4. Set up API for inference
5. Send a request and get a prediction



Step 1: Build model

```
import numpy as np
import onnxruntime as ort
import skl2onnx
import skl2onnx.common.data_types
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from fastapi import FastAPI
from pydantic import BaseModel

# Generate synthetic data
X = np.random.rand(100, 3)
y = 2*X[:, 0] + 3*X[:, 1] - X[:, 2] + np.random.normal(0, 0.1, 100)

# Create pipeline with scaling and linear regression
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('regressor', LinearRegression())
])

# Fit the pipeline
pipeline.fit(X, y)
```


Step 2: Convert model to ONNX

```
# Convert entire pipeline to ONNX
initial_type = [('input', skl2onnx.common.data_types.FloatTensorType([None, 3]))]
onx = skl2onnx.convert_sklearn(
    pipeline,
    initial_types=initial_type,
    target_opset=12
)

# Save ONNX model
with open("model.onnx", "wb") as f:
    f.write(onx.SerializeToString())
```

Step 3: Validate Input and model predictions

```
class PredictionInput(BaseModel):  
    features: list[float]  
  
class PredictionOutput(BaseModel):  
    prediction: float
```

Step 3: Validate Input and model predictions

```
# Verification function (optional)
def verify_pipeline_prediction():
    # Compare sklearn pipeline prediction with ONNX prediction
    test_input = np.random.rand(1, 3)

    # Sklearn pipeline prediction
    sklearn_pred = pipeline.predict(test_input)

    # ONNX prediction
    ort_inputs = {ort_session.get_inputs()[0].name: test_input.astype(np.float32)}
    onnx_pred = ort_session.run(None, ort_inputs)[0]

    print("Sklearn Prediction:", sklearn_pred)
    print("ONNX Prediction:   ", onnx_pred)
    print("Prediction Match:  ", np.allclose(sklearn_pred, onnx_pred))

# Optional: Run verification
verify_pipeline_prediction()
```

Step 4: Set up API for inference

```
# FastAPI app for model serving
app = FastAPI()

# Load ONNX model
ort_session = ort.InferenceSession("model.onnx")

@app.post("/predict")
async def predict(input_data: PredictionInput) -> PredictionOutput:
    # Prepare input for ONNX runtime
    input_name = ort_session.get_inputs()[0].name
    input_array = np.array([input_data.features], dtype=np.float32)

    # Get prediction (scaling and prediction happen in ONNX)
    prediction = ort_session.run(None, {input_name: input_array})[0][0]

    return PredictionOutput(prediction=float(prediction))
```

Step 5: Test it by sending requests to API

```
import requests

# URL of the FastAPI endpoint
url = "http://127.0.0.1:8000/predict"

# Sample input features (should match the 3 features used in training)
input_data = {
    "features": [0.5, 0.3, 0.7]
}

# Send POST request
response = requests.post(url, json=input_data)

# Print the prediction
print("Prediction:", response.json())
```

✓ 0.1s

Prediction: {'prediction': 1.2082703113555908}